

Logica Fuzzy: Approfondimenti Teorici e Ponte verso l'Implementazione

Complemento al corso *Computational Intelligence and Deep Learning*

Scopo del documento. Le tue slide coprono con solidità i fondamenti: fuzzy set, membership function, t-norm/t-conorm, relazioni fuzzy, inferenza (Mamdani, TSK, Singleton) e ANFIS. Questo documento non li ripete: si concentra sulle *lacune* che emergono quando si passa dalla teoria all'implementazione di progetti reali. Ogni sezione motiva *perché* un argomento serve, ne dà la formulazione matematica, e include lo scheletro di codice Python corrispondente.

Contents

1	Perché questi argomenti mancano (e perché contano)	1
2	Fuzzification e Defuzzification in dettaglio	1
2.1	Fuzzification	1
2.2	Defuzzification: le formule che servono	1
2.2.1	Centroid / Center of Area (COA)	2
2.2.2	Mean of Maxima (MOM)	2
2.2.3	Bisector, Smallest/Largest of Maxima	2
3	Fuzzy C-Means (FCM): clustering fuzzy	2
3.1	Formulazione	3
3.2	Dalla partizione fuzzy alle regole	3
4	Progettazione sistematica di un Fuzzy Inference System	3
4.1	Numero di membership function per variabile	3
4.2	Overlap e ε -completeness	4
4.3	Checklist di progetto	4
5	Interval Type-2 Fuzzy Systems (IT2 FLS)	4
5.1	L'idea	4
5.2	Perché conviene	4
6	Genetic-Fuzzy Systems: unire i due mondi	5
6.1	Cosa ottimizza il GA	5
6.2	Fitness	5
7	Metriche di valutazione di un sistema fuzzy	5
8	L'ecosistema Python per la logica fuzzy	6
8.1	scikit-fuzzy (skfuzzy) — la scelta di riferimento	6
8.2	Le altre opzioni e quando servono	6

1 Perché questi argomenti mancano (e perché contano)

Un corso universitario privilegia l'impianto formale: definisce gli oggetti (fuzzy set, operatori) e i teoremi (composizione, principio di estensione). Quello che tipicamente resta implicito è la *pipeline operativa* — come un valore numerico entra nel sistema, viene processato, ed esce come numero utilizzabile — e le tecniche che rendono un sistema fuzzy *apprendibile dai dati* anziché scritto a mano da un esperto. Le lacune che ho identificato, in ordine di priorità implementativa:

1. **Defuzzification** con formule operative e trade-off (le slide danno i nomi, non le formule).
2. **Fuzzy C-Means**: l'algoritmo di clustering che le slide citano per generare regole ANFIS ma non sviluppano.
3. **Progettazione sistematica di un FIS**: come scegliere numero di MF, overlap, granularità.
4. **Interval Type-2 Fuzzy Systems**: gestione dell'incertezza sulla membership stessa (una riga nelle slide).
5. **Genetic-Fuzzy Systems**: integrazione fra i due mondi che il corso tratta separatamente.
6. **Metriche di valutazione**: interpretabilità vs. accuratezza, RMSE, misure quantitative.

2 Fuzzification e Defuzzification in dettaglio

2.1 Fuzzification

La fuzzification mappa un valore crisp x_0 nei gradi di appartenenza ai fuzzy set antecedenti. Il metodo dominante è il **fuzzifier singleton**: x_0 diventa un fuzzy set il cui supporto è il solo punto x_0 con grado 1. Il motivo pratico è la semplificazione del *supremum* nella composizione: come mostrano le tue slide, con un singleton il sup collassa in una semplice valutazione $\mu_A(x_0)$. Alternative (fuzzifier gaussiano o triangolare centrato su x_0) si usano solo quando l'input è esso stesso rumoroso e si vuole modellarne l'incertezza.

2.2 Defuzzification: le formule che servono

Dopo aggregazione si ottiene un fuzzy set di output C con membership $\mu_C(y)$ sull'universo Y . La defuzzification lo riduce a un numero. I metodi principali:

2.2.1 Centroid / Center of Area (COA)

È il metodo di riferimento perché tiene conto dell'intera forma dell'aggregato:

$$y^* = \frac{\int_Y y \mu_C(y) dy}{\int_Y \mu_C(y) dy} \quad (\text{caso discreto: } y^* = \frac{\sum_k y_k \mu_C(y_k)}{\sum_k \mu_C(y_k)})$$

Pro: continuo, tiene conto di tutti i contributi. **Contro:** costoso, e può restituire un valore in una zona a bassa membership (il centro di gravità di due picchi separati cade nella valle tra loro).

2.2.2 Mean of Maxima (MOM)

Media dei punti a massima membership. Detto $M = \{y : \mu_C(y) = \max_{y'} \mu_C(y')\}$:

$$y^* = \frac{1}{|M|} \sum_{y \in M} y$$

Pro: economico, sceglie la zona di massima attivazione. **Contro:** ignora la forma; discontinuo (piccole variazioni di input possono far saltare y^*).

2.2.3 Bisector, Smallest/Largest of Maxima

Il *bisector* trova y^* che divide l'area in due metà uguali. *SOM/LOM* prendono il minimo/massimo dell'insieme dei massimi — utili in controllo quando si vuole una risposta conservativa o aggressiva.

Scelta pratica: COA per la maggior parte dei sistemi Mamdani (bilanciato). MOM quando serve velocità e la variabile di output è naturalmente “a modalità”. In un sistema TSK la defuzzification è già implicita nella media pesata degli output lineari, quindi il problema non si pone.

```

1 import numpy as np
2
3 def centroid(y, mu):
4     """Defuzzification COA su universo discreto y con membership mu."""
5     denom = np.sum(mu)
6     return np.sum(y * mu) / denom if denom > 0 else 0.0
7
8 def mean_of_maxima(y, mu):
9     peak = np.max(mu)
10    return np.mean(y[np.isclose(mu, peak)])

```

3 Fuzzy C-Means (FCM): clustering fuzzy

Le tue slide, parlando di ANFIS, dicono che “si può usare un algoritmo di clustering per trovare i centri”. Quell'algoritmo, nella pratica, è quasi sempre **Fuzzy C-Means**. È il ponte tra la logica fuzzy e l'apprendimento non supervisionato: mentre nel k -means ogni punto appartiene a un solo cluster, in FCM ogni punto appartiene a *tutti* i cluster con gradi $u_{ij} \in [0, 1]$, coerentemente con la filosofia fuzzy.

3.1 Formulazione

Dati N punti $\mathbf{x}_j$ e C cluster con centri $\mathbf{v}_i$, FCM minimizza:

$$J_m = \sum_{j=1}^N \sum_{i=1}^C u_{ij}^m \|\mathbf{x}_j - \mathbf{v}_i\|^2, \quad \sum_{i=1}^C u_{ij} = 1 \quad \forall j$$

dove $m > 1$ è il **fuzzifier** (tipicamente $m = 2$): controlla quanto “morbida” è l'assegnazione. Per $m \rightarrow 1$ si degenera nel k -means; per m grande le membership tendono tutte a $1/C$. La minimizzazione alternata dà le due update:

$$u_{ij} = \left(\sum_{k=1}^C \left(\frac{\|\mathbf{x}_j - \mathbf{v}_i\|}{\|\mathbf{x}_j - \mathbf{v}_k\|} \right)^{\frac{2}{m-1}} \right)^{-1}, \quad \mathbf{v}_i = \frac{\sum_j u_{ij}^m \mathbf{x}_j}{\sum_j u_{ij}^m}$$

3.2 Dalla partizione fuzzy alle regole

Proiettando i centri $\mathbf{v}_i$ sugli assi delle variabili di input si ottengono i centri delle membership function antecedenti; ogni cluster diventa una regola. Questo genera un FIS *data-driven* senza dover posizionare le MF a mano — l'alternativa al grid partitioning che le slide descrivono per ANFIS.

Quando usare grid vs. clustering. Grid partitioning (MF equispaziate) massimizza l'interpretabilità ma soffre la “maledizione della dimensionalità”: g MF per n input danno g^n

regole. Il clustering scala molto meglio con n , al costo di regole meno leggibili (ogni regola ha i propri fuzzy set). Regola pratica: grid fino a 2–3 input, clustering oltre.

```

1 import numpy as np
2
3 def fuzzy_cmeans(X, C, m=2.0, max_iter=150, tol=1e-5):
4     N = X.shape[0]
5     U = np.random.dirichlet(np.ones(C), size=N) # righe sommano a 1
6     for _ in range(max_iter):
7         Um = U ** m
8         V = (Um.T @ X) / Um.sum(axis=0)[:, None] # centri
9         dist = np.linalg.norm(X[:, None, :] - V[None, :, :], axis=2)
10        dist = np.fmax(dist, 1e-12)
11        U_new = 1.0 / np.sum(
12            (dist[:, :, None] / dist[:, None, :]) ** (2/(m-1)), axis=2)
13        if np.linalg.norm(U_new - U) < tol:
14            break
15        U = U_new
16    return V, U
17 # NB: skfuzzy.cmeans() lo implementa già pronto all'uso.

```

4 Progettazione sistematica di un Fuzzy Inference System

Le slide trattano la scelta dei parametri come “empirica / per tentativi”. Ci sono però principi consolidati che vale la pena conoscere.

4.1 Numero di membership function per variabile

Tipicamente 3–7 etichette (es. *low/medium/high*). Meno di 3 non cattura la non-linearità; più di 7 supera la risoluzione percettiva umana e riduce l’interpretabilità. La granularità va commisurata a quanto la variabile influisce sull’output.

4.2 Overlap e ε -completeness

Due MF adiacenti devono sovrapporsi affinché ogni input attivi almeno una regola con grado sufficiente. La proprietà di ε -**completeness** richiede che

$$\forall x \in X \quad \exists A_i : \mu_{A_i}(x) \geq \varepsilon,$$

con $\varepsilon \approx 0.5$ come valore comune (le tue slide citano $\varepsilon = 0.05$ nel contesto ANFIS, che è la soglia minima per evitare “buchi” di copertura). Overlap tipico: il crossover tra due MF triangolari adiacenti cade a $\mu = 0.5$.

4.3 Checklist di progetto

1. Definire universo del discorso di ogni variabile (range fisico).
2. Scegliere numero e forma delle MF (triangolari/trapezoidali per semplicità; gaussiane/bell se serve derivabilità per training).
3. Garantire copertura (ε -completeness) e overlap ragionevole.
4. Scrivere la rule base (Mamdani se serve interpretabilità, TSK se serve accuratezza numerica).
5. Scegliere t-norm (AND), t-conorm (OR), metodo di implicazione, aggregazione, defuzzifier.
6. Validare su dati; se disponibili, ottimizzare i parametri (ANFIS o GA).

5 Interval Type-2 Fuzzy Systems (IT2 FLS)

Le slide menzionano i fuzzy set di tipo 2 in una riga. Meritano di più perché rappresentano il principale filone “avanzato” della logica fuzzy e sono ottimi per un progetto che si distingue.

5.1 L'idea

In un fuzzy set classico (type-1) il grado di appartenenza $\mu_A(x)$ è un numero preciso. Ma se l'esperto non è sicuro di *quale* membership function usare — “alto” significa 180 cm o 185 cm? — allora la membership stessa è incerta. Un **type-2 fuzzy set** modella questa incertezza: $\mu_A(x)$ diventa a sua volta un fuzzy set (in $[0, 1]$).

Nel caso più trattabile, l'**interval type-2**, la membership secondaria è uniforme su un intervallo. L'insieme delle membership possibili forma la **Footprint of Uncertainty (FOU)**, delimitata da una MF superiore $\bar{\mu}_A(x)$ e una inferiore $\underline{\mu}_A(x)$.

5.2 Perché conviene

Un IT2 FLS gestisce meglio rumore e incertezza dei parametri: dove un type-1 dà una risposta netta ma fragile, il type-2 dà una risposta più robusta. Il prezzo è un passo aggiuntivo, il **type reduction** (tipicamente l'algoritmo iterativo di Karnik-Mendel), che collassa l'output intervallare in un type-1 prima della defuzzification.

Per il portfolio: implementare un controller IT2 e confrontarlo con l'equivalente type-1 sotto rumore crescente è un sottoprogetto che dimostra padronanza oltre il livello del corso. La libreria `pyit2fls` copre questo caso.

6 Genetic-Fuzzy Systems: unire i due mondi

Il corso tratta algoritmi genetici (GA) e sistemi fuzzy separatamente. La loro integrazione — i **Genetic Fuzzy Systems (GFS)** — è il naturale sistema ibrido, e sfrutta esattamente l'osservazione delle tue slide secondo cui un GA “può essere usato per ottimizzare opportunamente alcuni parametri”.

6.1 Cosa ottimizza il GA

In un FIS ci sono due tipi di parametri ottimizzabili:

- **Tuning dei parametri** (approccio più comune): il cromosoma codifica i parametri delle MF (centri, ampiezze). La rule base resta fissa. È il caso più semplice e stabile.
- **Learning della rule base**: il cromosoma codifica *quali* regole includere e i loro conseguenti. Più potente, più difficile.

6.2 Fitness

Per un problema di regressione/controllo la fitness è tipicamente l'inverso (o il negativo) dell'errore sul training set:

$$\text{fitness} = -\text{RMSE} = -\sqrt{\frac{1}{N} \sum_{j=1}^N (y_j - \hat{y}_j)^2}$$

Un termine di penalità sul numero di regole o sull'overlap può essere aggiunto per preservare l'interpretabilità (ottimizzazione multi-obiettivo: accuratezza *vs.* leggibilità).

GFS vs. ANFIS. ANFIS usa gradient descent + least squares e richiede MF derivabili e una struttura TSK; converge veloce ma può incastrarsi in minimi locali. Il GA non richiede derivabilità, esplora globalmente, e può ottimizzare anche la struttura delle regole; è però più lento. Sono complementari: un pattern comune è GA per la struttura + gradient per il fine-tuning.

7 Metriche di valutazione di un sistema fuzzy

Un aspetto assente dalle slide ma cruciale per un progetto serio: come si *misura* la qualità di un FIS.

- **Accuratezza** (regressione): RMSE, MAE, R^2 tra output previsto e desiderato.
- **Accuratezza** (classificazione): accuracy, precision/recall, matrice di confusione — valutando la classe come argmax delle membership di output.
- **Interpretabilità**: numero di regole, numero di MF per variabile, grado di overlap, “distinguibilità” delle etichette linguistiche. Esistono indici formali (es. l’indice di Nauck) che combinano queste misure.
- **Complessità**: numero di parametri, tempo di inferenza — rilevante per sistemi embedded/real-time, dove i FIS eccellono.

Il compromesso **accuratezza–interpretabilità** è il tema centrale della ricerca sui sistemi fuzzy: Mamdani sta sul lato interpretabile, TSK sul lato accurato, e Singleton (come notano le slide) è il compromesso.

8 L’ecosistema Python per la logica fuzzy

Panoramica delle tecnologie con le rispettive motivazioni d’uso.

8.1 scikit-fuzzy (skfuzzy) — la scelta di riferimento

È la libreria più usata e stabile per la logica fuzzy in Python, parte dello stack SciPy. Fornisce membership function, t-norm/t-conorm, un `ControlSystem` di alto livello per costruire FIS Mamdani, e `cmeans` per il Fuzzy C-Means. **Motivazione:** è il default sensato per iniziare — API matura, documentazione con esempi, integrazione naturale con NumPy. Nota: la manutenzione è a bassa cadenza (ultima release maggiore 0.5.0 nel 2024, compatibile con Python 3.x moderno), ma il core è stabile e completo.

8.2 Le altre opzioni e quando servono

- **simpy** — API più moderna e leggibile, supporta Mamdani e Sugeno/TSK nativamente. Motivazione: quando vuoi codice più pulito o hai bisogno di TSK, che skfuzzy non gestisce elegantemente.
- **pyit2fls** — specializzata negli Interval Type-2 FLS. Motivazione: unica scelta pratica se il progetto è sui sistemi di tipo 2.
- **scikit-learn** — non fa logica fuzzy, ma è indispensabile per il contorno: split train/test, cross-validation, metriche, e per confrontare il tuo FIS con baseline (SVM, random forest). Motivazione: qualsiasi progetto ML serio la usa per la valutazione.

- **DEAP** — framework per algoritmi evolutivi/genetici. Motivazione: se implementi un Genetic-Fuzzy System e non vuoi scrivere il GA da zero.
- **TensorFlow / PyTorch** — per ANFIS “moderno” implementato come rete derivabile e allenato con autograd. Motivazione: quando vuoi ANFIS su dataset grandi con accelerazione GPU, o integrarlo in una pipeline deep learning.
- **NumPy puro** — implementare gli algoritmi a mano (come i frammenti in questo documento). Motivazione: è il modo migliore per *imparare* e per un portfolio che mostri comprensione, non solo uso di librerie.

Strategia consigliata per il repository: scikit-fuzzy come base stabile per i progetti principali, con *mini-test* dedicati che provano *simpful* (per il TSK), *pyit2fls* (per il type-2) e un'implementazione NumPy-from-scratch (per dimostrare padronanza). Questo copre sia stabilità sia ampiezza, ed è esattamente ciò che rende un portfolio interessante.